1. Setup and agent configuration

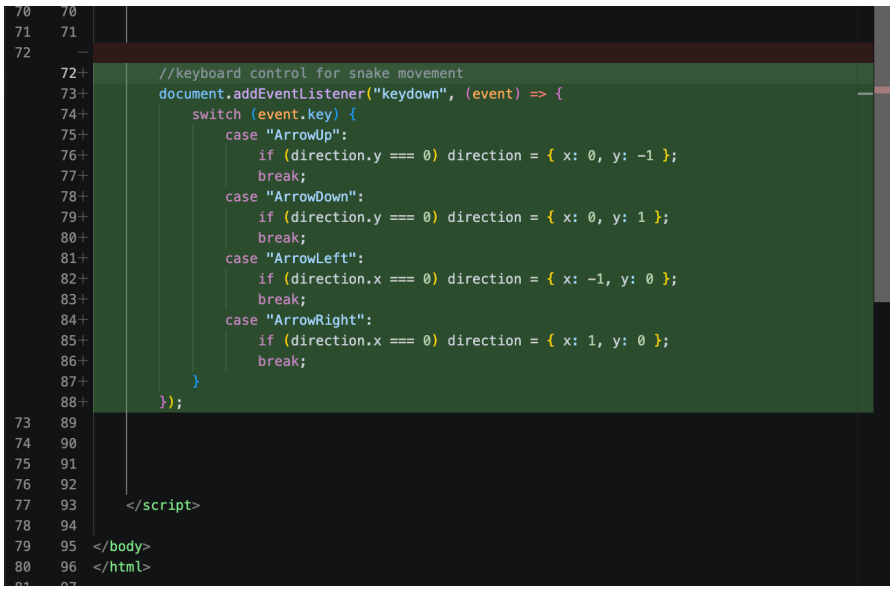
I set up the GitHub copliot through Vsvode extensions. From there you can download the copliot and vs agent to help you. It was somehwat easily done, thankfully. To the starter agent filkes I made minor changes which were to help me understand the code rather than to only split the code for me.

2. Development process

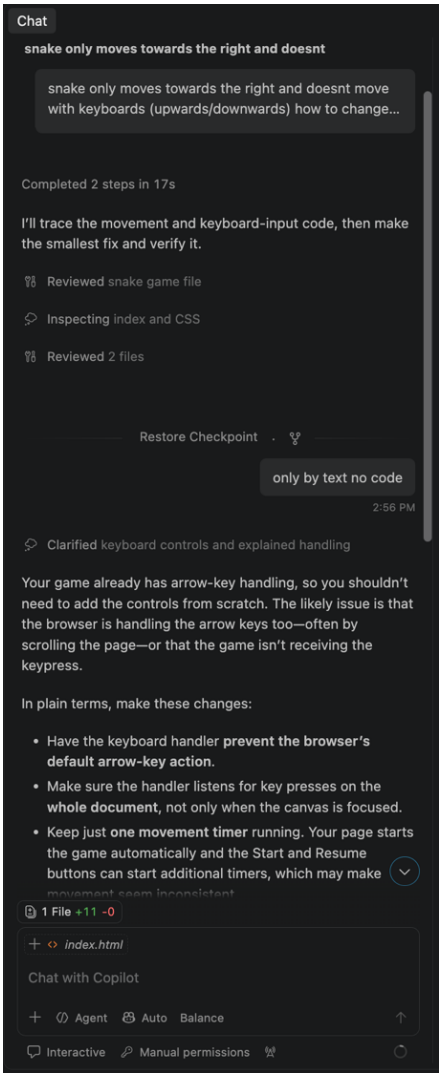
(1)

First important moment was probably adding my first code which was the keyboard control for the snake. With my code the user could use their own keyboards sich as upwards, downwards to move the snake towards the food.

When I had problems with the snake not moving in certain areas or in ways I simply wasn't pleased with, I asked my agent for assistance. With its help, I was able to incorporate those changes.



```
70  70
71  71
72
    72+     //keyboard control for snake movement
    73+     document.addEventListener("keydown", (event) => {
    74+         switch (event.key) {
    75+             case "ArrowUp":
    76+                 if (direction.y === 0) direction = { x: 0, y: -1 };
    77+                 break;
    78+             case "ArrowDown":
    79+                 if (direction.y === 0) direction = { x: 0, y: 1 };
    80+                 break;
    81+             case "ArrowLeft":
    82+                 if (direction.x === 0) direction = { x: -1, y: 0 };
    83+                 break;
    84+             case "ArrowRight":
    85+                 if (direction.x === 0) direction = { x: 1, y: 0 };
    86+                 break;
    87+         }
    88+     });
73  89
74  90
75  91
76  92
77  93      </script>
78  94
79  95  </body>
80  96  </html>
```



(2)

The second memorable moment occurred when coding the logic for the game to end if the snake hit a wall or itself. The reason I was having a hard time was that the snake would hit the wall before I could even move it anywhere. Because of that, the game would abruptly end. At that time, I consulted the AI agent, and it gave me a few ideas on how to adjust and rewrite my code.

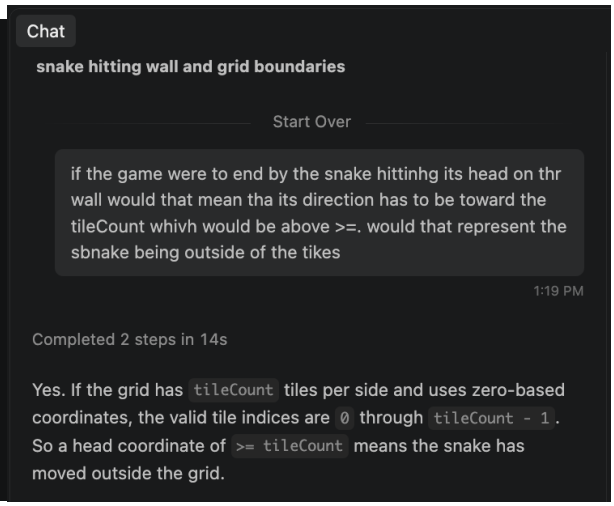


```
//game over segment where:

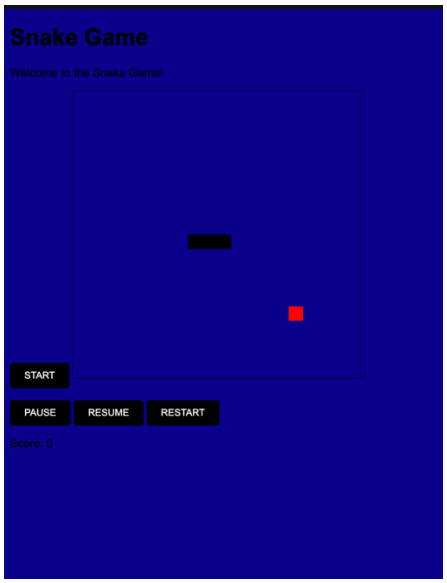

//hitting the wall = ends the game
const checkGameOver = (head) => {
    const hitWall = head.x < 0 || head.x >= tileCount || head.y < 0 || head.y >= tileCount;
    if (hitWall) {
        clearInterval(gameInterval);
        alert("Game Over! Your score: " + score);
        return true;
    }
    return false;
}

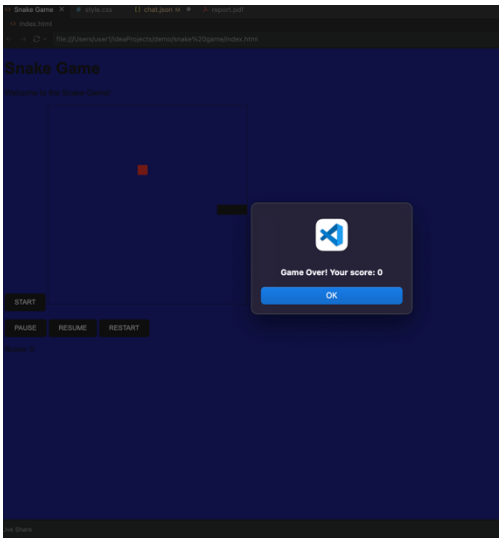
//if snake hits itself = ends the game
  const checkHitSelf = (head, body = snake) => {
    const hitSelf = body.some(segment => segment.x === head.x && segment.y === head.y);
    if (hitSelf) {
        clearInterval(gameInterval);
        alert("Game Over! Your score: " + score);
        return true;
    }

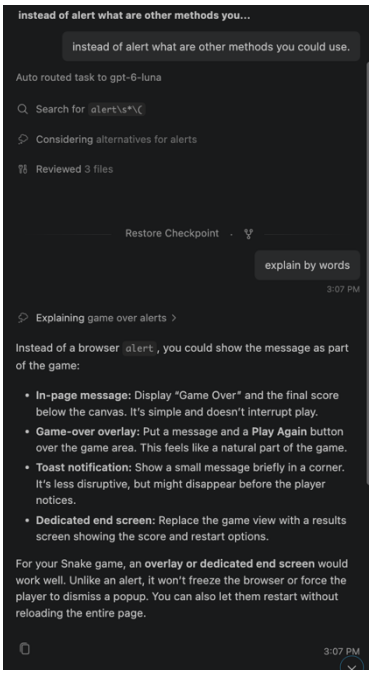
    return false;
};
```



(3) In this moment, I was finalizing my project, which meant I was looking at different ways to make it look more aesthetically pleasing. Because I wasn't the biggest fan of the alert popup, I asked the AI agent to suggest other methods that would be better suited for the project. I implemented the suggestions I received from the agent while also utilizing the skills I already had.







3. Reflection

During this project, the AI agent was a lot of help. It guided me through harder concepts and assisted me with my ideas. I utilized a lot of the HTML and CSS teachings from this course, as well as JavaScript. I'd like to think I used the main concepts that were part of this course. For me, the newer concepts were perhaps related to JSON. I had a hard time understanding the exportation, and it took a while to grasp. Also, using Git was difficult, especially as a new user. Understanding how to commit, add comments, and check on GitHub that everything worked was a lot to take in, but the more I did it, the more I understood it.

I noticed that the chat's responses and the inline code suggestions were mainly aligned. If the chat advised me to try a method, majority of the time, after writing a little bit of my own code, the inline assistant would suggest something to add to it. Although there were times it would give rather false information or ideas that weren't part of the project, I find it to be the developer's responsibility to understand the code enough to know if the inline suggestions are valuable or not. For instance, there were parts during the snake collision logic where it gave wrong suggestions, and I had to figure out on my own where to go from there.

In the future, I would make sure to be more specific with the instructions I give, so that it wouldn't just give me straight code answers. Instead, I would ask it to provide instructions and advice so that I, as the developer, can understand more of my own code. I wouldn't position the agent anywhere else than where it was before, simply because custom agents are meant to help rather than to be fully automated in production. I believe that a project should perhaps have one agent to help with smaller tasks. The programmer's job is to make sure they understand what is happening inside their code and why.